

Faculdade Cotemig

Disciplina: Fundamentos Teóricos da Computação

Trabalho Final

*Implementação de Máquinas Abstratas:
AFD, Autômato de Pilha e Máquina de
Turing*

Professor: Júlio César da Silva
Atividade em Grupo: No mínimo 2 e no máximo 3 aluno(a)s.
Data de Entrega: 10/06/2026

Valor Total: 10,0 pontos

Sumário

1	Apresentação e Objetivos	2
1.1	Grupo de aluno(a)s:	2
1.2	Competências Avaliadas	2
2	Contextualização Teórica	3
2.1	Autômato Finito Determinístico (AFD)	3
2.2	Autômato de Pilha com Reconhecimento por Pilha Vazia	3
2.3	Máquina de Turing	4
3	Especificação das Tarefas	4
3.1	Parte 1 — Implementação do AFD (2,5 pontos)	4
3.2	Parte 2 — Implementação do Autômato de Pilha (3,0 pontos)	5
3.3	Parte 3 — Implementação da Máquina de Turing (3,0 pontos)	7
3.4	Parte 4 — Relatório Técnico (1,5 ponto)	8
4	Estrutura de Entrega	8
4.1	Repositório GitHub	9
4.2	Vídeo de Defesa	10
5	Rubrica de Avaliação	11
6	Orientações Finais e Dicas	15

1. Apresentação e Objetivos

Este trabalho final tem como objetivo avaliar a capacidade dos alunos de **compreender, modelar e implementar** três máquinas abstratas fundamentais da teoria da computação, utilizando a linguagem de programação **C#**:

1. **Autômato Finito Determinístico (AFD)** — reconhecedor de linguagens regulares;
2. **Autômato de Pilha (AP) com reconhecimento por pilha vazia** — reconhecedor de linguagens livres de contexto;
3. **Máquina de Turing (MT)** — modelo universal de computação.

1.1. Grupo de aluno(a)s:

- Grupo de no mínimo 2 e no máximo 3 aluno(a)s;
- TODOS OS PARTICIPANTES devem enviar o link do trabalho na respectiva atividade no Google Sala de Aula.

1.2. Competências Avaliadas

- Compreensão formal das definições das três máquinas abstratas;
- Capacidade de traduzir formalismos matemáticos em código funcional;
- Qualidade da documentação e justificativas de projeto;
- Raciocínio crítico sobre os limites de cada modelo computacional;
- Comunicação técnica durante o vídeo de defesa.

Uso de Ferramentas de IA e Integridade Acadêmica

O uso de ferramentas de inteligência artificial (ChatGPT, Copilot, Gemini, entre outras) é **permitido como recurso de consulta e apoio** ao estudo, da mesma forma que livros, tutoriais e fóruns especializados. Espera-se, contudo, que o código e o relatório entregues sejam resultado do esforço intelectual da equipe.

Critério de originalidade: trabalhos com similaridade superior a **60%** em relação a outro trabalho da turma ou a código gerado integralmente por IA serão enquadrados como **plágio e receberão nota zero**, sem possibilidade de reapresentação. Os trabalhos poderão ser submetidos a ferramentas de detecção de similaridade a critério do professor.

Este trabalho exige a entrega de um vídeo de defesa (ver Seção 4): cada equipe deverá demonstrar a execução do programa e explicar as escolhas de implementação. Inconsistências evidentes entre o vídeo e o código entregue poderão resultar em anulação da nota.

2. Contextualização Teórica

As seções a seguir sintetizam os formalismos que **devem guiar a implementação**. Esperamos que o código seja uma tradução direta da definição matemática, não uma solução empírica encontrada na internet.

2.1. Autômato Finito Determinístico (AFD)

Definição Formal — AFD

Um **AFD** é uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$ onde:

- Q é um conjunto finito de **estados**;
- Σ é um **alfabeto** finito de entrada;
- $\delta : Q \times \Sigma \rightarrow Q$ é a **função de transição**;
- $q_0 \in Q$ é o **estado inicial**;
- $F \subseteq Q$ é o conjunto de **estados de aceitação**.

Uma cadeia $w \in \Sigma^*$ é **aceita** se a computação de M sobre w termina em um estado $q \in F$.

2.2. Autômato de Pilha com Reconhecimento por Pilha Vazia

Definição Formal — AP (pilha vazia)

Um **AP** é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$ onde:

- Q é um conjunto finito de estados;
- Σ é o alfabeto de entrada;
- Γ é o **alfabeto da pilha**;
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ é a função de transição;
- $q_0 \in Q$ é o estado inicial;
- $Z_0 \in \Gamma$ é o **símbolo inicial da pilha**;
- O conjunto de estados de aceitação é $\emptyset$ (aceitação por **pilha vazia**).

Uma cadeia w é aceita se existe uma computação de M que, ao consumir w , esvazia completamente a pilha.

2.3. Máquina de Turing

Definição Formal — MT

Uma **Máquina de Turing** é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ onde:

- Q é o conjunto finito de estados;
- Σ é o alfabeto de entrada ($\sqcup \notin \Sigma$);
- $\Gamma \supseteq \Sigma$ é o alfabeto da fita ($\sqcup \in \Gamma$);
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição;
- q_0 é o estado inicial;
- q_{accept} é o estado de aceitação;
- q_{reject} é o estado de rejeição ($q_{\text{reject}} \neq q_{\text{accept}}$).

3. Especificação das Tarefas

O trabalho é composto por **três partes independentes** e um **relatório técnico**. Cada parte deve ser implementada em C# (.NET 6 ou superior).

3.1. Parte 1 — Implementação do AFD (2,5 pontos)

Tarefa 1 — Simulador Genérico de AFD

Linguagem-alvo:

$$L_1 = \{ w \in \{a, b\}^* \mid w \text{ termina com } ab \}$$

Exigências de implementação:

- Represente o AFD como a 5-tupla formal $(Q, \Sigma, \delta, q_0, F)$ em estruturas de dados C# explicitamente nomeadas conforme a definição matemática. **Não utilize bibliotecas externas de autômatos.**
- Implemente a função de transição δ como um `Dictionary<(string estado, char simbolo), string>`.
- Implemente o método `bool Aceitar(string cadeia)` que simula a leitura da cadeia símbolo a símbolo.
- O programa deve ler múltiplas cadeias de um arquivo de texto (`entradas.txt`), uma por linha, e exibir para cada uma: a cadeia, o rastro de estados percorridos e o resultado (*ACEITA* ou *REJEITA*).
- Implemente um método `void ExibirDiagrama()` que imprime no console uma representação textual da tabela de transições do AFD.
- Desafio obrigatório:** além do AFD para L_1 , o programa deve ser capaz de

carregar qualquer AFD a partir de um arquivo de configuração `afd.json` com o seguinte esquema:

- Campos: `estados`, `alfabeto`, `estadoInicial`, `estadosAceitacao`, `transicoes` (lista de objetos com `origem`, `simbolo`, `destino`).

Casos de teste obrigatórios a incluir em `entradas.txt`:

Cadeia	Esperado	Justificativa
ab	ACEITA	Caso mínimo
aab	REJEITA	Não termina com ab
bab	ACEITA	Prefixo irrelevante
ababab	ACEITA	Múltiplas ocorrências
ba	REJEITA	Termina com a apenas
ε (cadeia vazia)	REJEITA	Sem caracteres
b	REJEITA	Um símbolo

Questões Reflexivas — Parte 1

Inclua no relatório as respostas às seguintes perguntas (mínimo de 8 linhas cada):

1. Por que a linguagem L_1 é regular? Apresente a expressão regular equivalente.
2. Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado.
3. O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas (símbolos fora do alfabeto)?

3.2. Parte 2 — Implementação do Autômato de Pilha (3,0 pontos)

Tarefa 2 — Simulador de AP com Pilha Vazia

Linguagem-alvo:

$$L_2 = \{ a^n b^n \mid n \geq 1 \}$$

Exigências de implementação:

- a) Represente o AP como a 7-tupla formal. A pilha deve ser implementada manualmente como uma `Stack<char>` do C# (não use `LinkedList` ou arrays).
- b) A aceitação deve ser **exclusivamente por pilha vazia** — não é permitido verificar estado final.
- c) Implemente transições com λ -movimentos (representados como `'\0'` no código).

- d) O método de simulação deve exibir, a cada passo, o **conteúdo atual da pilha**, o estado corrente e o restante da cadeia de entrada (configuração instantânea).
- e) Leia as cadeias do arquivo `entradas_ap.txt` e exiba os resultados.
- f) **Desafio obrigatório:** adicione um segundo AP que reconheça a linguagem

$$L_3 = \{ w \in \{a, b\}^* \mid w = w^R, |w| \geq 1 \}$$

(palíndromos sobre $\{a, b\}$) por pilha vazia. Justifique a diferença de complexidade entre L_2 e L_3 .

Casos de teste para L_2 :

Cadeia	Esperado	Justificativa
ab	ACEITA	$n = 1$
aabb	ACEITA	$n = 2$
aaabbb	ACEITA	$n = 3$
aab	REJEITA	$2a, 1b$
abb	REJEITA	$1a, 2b$
ba	REJEITA	Ordem invertida
ε	REJEITA	$n \geq 1$
abab	REJEITA	Intercalado

Casos de teste para L_3 (palíndromos):

Cadeia	Esperado
a	ACEITA
aba	ACEITA
abba	ACEITA
ab	REJEITA
aab	REJEITA

Questões Reflexivas — Parte 2

1. Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento para linguagens regulares.
2. A aceitação por pilha vazia e por estado final são equivalentes em poder de reconhecimento? Demonstre conceitualmente.
3. Explique o papel do símbolo Z_0 na sua implementação.

3.3. Parte 3 — Implementação da Máquina de Turing (3,0 pontos)

Tarefa 3 — Simulador de Máquina de Turing

Linguagem-alvo:

$$L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$$

Exigências de implementação:

- Implemente a fita como uma estrutura dinâmica (ex.: `Dictionary<int,char>` onde a chave é a posição). O espaço em branco ($\sqcup$) deve ser representado por `'_'`.
- Implemente a função de transição δ como um `Dictionary<(string estado, char simbolo), (string novoEstado, char novoSimbolo, char direcao)>`.
- A simulação deve exibir a cada passo:
 - O estado atual;
 - O conteúdo completo da fita (com delimitadores `[]` ao redor do símbolo sob o cabeçote);
 - A posição do cabeçote.
- A MT deve ter estados explícitos para q_{accept} e q_{reject} .
- Implemente um **contador de passos** e um **limite configurável** de passos para evitar loops infinitos durante os testes.
- Desafio obrigatório:** implemente uma segunda MT que **compute** (não apenas reconheça) a função $f(n) = n + 1$ em unário (representação: n ocorrências do símbolo 1). Ao final, a fita deve conter exatamente $n + 1$ ocorrências de 1. Exemplo: entrada 111 $\rightarrow$ saída 1111.

Casos de teste para L_4 :

Cadeia	Esperado	Justificativa
abc	ACEITA	$n = 1$
aabbcc	ACEITA	$n = 2$
aaabbbccc	ACEITA	$n = 3$
aabbc	REJEITA	Quantidades distintas
ab	REJEITA	Sem c
abc abc	REJEITA	Símbolos fora de ordem
ε	REJEITA	$n \geq 1$

Casos de teste para $f(n) = n + 1$ (unário):

Fita de entrada	Fita de saída esperada
1	11
111	1111
11111	111111

Questões Reflexivas — Parte 3

1. Por que L_4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento?
2. Quantos estados foram necessários para a MT de L_4 ? Descreva a estratégia de “marcação” adotada.
3. Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing.

3.4. Parte 4 — Relatório Técnico (1,5 ponto)

Relatório Técnico — Diretrizes

O relatório deve conter as seguintes seções, nessa ordem:

1. **Introdução** ($\frac{1}{2}$ página): contextualize o trabalho e os objetivos.
2. **Fundamentação Teórica** (1–2 páginas): apresente as definições formais das três máquinas, com exemplos de cálculo manual para as cadeias de teste.
3. **Descrição da Implementação** (2–3 páginas): descreva as decisões de projeto, as estruturas de dados escolhidas e como elas se relacionam com a definição formal.
4. **Resultados e Testes** (1 página): apresente tabelas com os resultados obtidos para todos os casos de teste.
5. **Análise Comparativa** ($\frac{1}{2}$ –1 página): compare o poder computacional das três máquinas, abordando as linguagens que cada uma pode (e não pode) reconhecer.
6. **Conclusão** ($\frac{1}{2}$ página): reflexões sobre o aprendizado obtido.
7. **Referências**: mínimo de 3 referências bibliográficas (livros ou artigos científicos).

Formatação: fonte Times New Roman ou Arial, tamanho 12, espaçamento 1,5, margens ABNT (superior e esquerda 3 cm, inferior e direita 2 cm).

4. Estrutura de Entrega

4.1. Repositório GitHub

Regras para o Repositório GitHub

Todo o código-fonte do trabalho **deve ser hospedado em um repositório público no GitHub**, criado especificamente para esta atividade. As regras a seguir são **obrigatórias**:

1. **Nome do repositório:** seguir o padrão `ftc-20261-final`.
2. **Estrutura de diretórios obrigatória:**
 - `/Parte1/` — projeto C# completo do AFD;
 - `/Parte2/` — projeto C# completo do Autômato de Pilha;
 - `/Parte3/` — projeto C# completo da Máquina de Turing;
 - `/docs/relatorio.pdf` — relatório técnico;
 - `README.md` — na raiz do repositório (ver regras abaixo).
3. **Arquivo `README.md` obrigatório** na raiz, contendo:
 - Nome completo e matrícula de todos os integrantes da equipe;
 - Breve descrição de cada parte implementada;
 - Instruções claras para compilar e executar cada projeto (`dotnet run` em ambiente .NET 6+);
 - Link para o vídeo de defesa (ver Seção 4.2).
4. **Commits incrementais:** o histórico de commits deve refletir o desenvolvimento gradual do trabalho. Repositórios com apenas um commit contendo todo o código serão penalizados em **0,5 ponto**, pois indicam ausência de desenvolvimento incremental.
5. **Mensagens de commit** devem ser descritivas e em português ou inglês. Exemplos adequados: "Implementa classe AFD com função de transição" ou "Add MT tape display step-by-step". Mensagens genéricas como "update" ou "fix" sem contexto serão desconsideradas na avaliação do histórico.
6. **Arquivo `.gitignore`:** é obrigatória a inclusão de um `.gitignore` adequado para projetos C#/.NET, evitando o versionamento de pastas `bin/`, `obj/` e arquivos temporários de IDE.
7. **Entrega via AVA:** submeta no Google Sala de Aula **apenas o link** do repositório GitHub. Não é necessário enviar arquivo `.zip`.
8. **Colaboradores:** em equipes, todos os integrantes devem ter contribuído com ao menos um commit. Integrantes sem nenhum commit no histórico poderão ter sua nota individualizada a critério do professor.

4.2. Vídeo de Defesa

Regras para o Vídeo de Defesa

Em substituição à apresentação presencial, cada equipe deverá produzir e disponibilizar um **vídeo de defesa** com as seguintes especificações:

1. **Duração:** mínimo de **10 minutos** e máximo de **20 minutos**. Vídeos fora desse intervalo serão penalizados em até 0,5 ponto.
2. **Conteúdo obrigatório do vídeo:**
 - **Apresentação da equipe:** cada integrante deve se identificar com nome e a parte pela qual foi responsável;
 - **Explicação dos algoritmos:** para cada uma das três partes, o(s) responsável(is) deve(m) explicar oralmente a lógica de implementação, relacionando o código à definição formal da máquina;
 - **Execução ao vivo:** demonstrar a execução do programa para ao menos **3 casos de teste por parte**, exibindo a saída no console e comentando os resultados;
 - **Discussão das questões reflexivas:** ao menos uma questão reflexiva de cada parte deve ser respondida verbalmente durante o vídeo.
3. **Formato técnico:**
 - O vídeo deve incluir **gravação de tela** (screencast) mostrando o código e a execução;
 - Incluir câmera do apresentador;
 - Qualidade mínima de áudio e vídeo que permita compreensão clara do conteúdo.
 - Cada membro da equipe deve apresentar um dos algoritmos.
4. **Hospedagem e compartilhamento:** o vídeo deve ser publicado no **YouTube** (modo não listado ou público). O link deve constar no **README.md** do repositório GitHub.
5. **Prazo:** o vídeo deve estar disponível no momento da submissão do link do repositório no Google Sala de Aula.

Prazos e Penalidades

- **Data de entrega (link GitHub + vídeo) via Google Sala de Aula:** 10/06/2026.
- **Vídeo ausente ou inacessível:** desconto de 5 pontos.
- **Repositório privado ou inacessível na data de avaliação:** nota 0.

5. Rubrica de Avaliação

Como ler a rubrica

Cada critério é pontuado em quatro níveis: **Excelente**, **Satisfatório**, **Insuficiente** e **Não Realizado**. A nota final é a soma das pontuações de todos os critérios, limitada a 10,0.

Rubrica — Parte 1: Autômato Finito Determinístico (2,5 pontos)

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
1.1 Fidelidade à 5-tupla formal Estruturas nomeadas conforme a definição matemática; cada campo tem semântica correta.	5-tupla completa, corretamente tipada e documentada. (0,5)	5-tupla presente mas com pequenas inconsistências na tipagem ou nomenclatura. (0,3)	Apenas parte da 5-tupla implementada ou com erros conceituais. (0,1)	Ausente. (0)
1.2 Corretude da simulação Todos os casos de teste produzem o resultado esperado.	100% dos casos corretos, rastro exibido. (0,6)	70–99% dos casos corretos. (0,4)	Menos de 70% dos casos corretos. (0,2)	Não executa. (0)
1.3 Leitura de arquivo e carga dinâmica Carrega qualquer AFD do arquivo de configuração.	Carrega, valida e exibe diagrama corretamente. (0,5)	Carrega mas sem validação ou diagrama incompleto. (0,3)	Parsing parcial com erros. (0,1)	Ausente. (0)
1.4 Qualidade do código Legibilidade, comentários, tratamento de exceções.	Código limpo, comentado, sem código morto. (0,4)	Código legível com comentários insuficientes. (0,25)	Código difícil de ler, sem comentários. (0,1)	(0)

(continua)

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
1.5 Questões reflexivas no relatório	Respostas completas, com	Respostas corretas mas	Respostas superficiais ou	Ausente.
Profundidade e correção das respostas.	prova formal e exemplos. (0,5)	sem profundidade formal. (0,3)	com erros. (0,1)	(0)
Subtotal máximo — Parte 1:				2,5

Rubrica — Parte 2: Autômato de Pilha (3,0 pontos)

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
2.1 Aceitação por pilha vazia	Correto, sem uso de estados finais. (0,6)	Correto mas usa flag auxiliar desnecessária. (0,4)	Mescla aceitação por estado e por pilha. (0,2)	Não implementa por pilha vazia. (0)
Nenhum estado de aceitação; aceita somente quando pilha fica vazia.				
2.2 Exibição de configurações instantâneas	Formato claro, todos os campos corretos. (0,5)	Exibe configuração mas com formatação inconsistente. (0,3)	Exibe apenas parte da configuração. (0,1)	Ausente. (0)
Cada passo mostra estado, entrada restante e pilha.				
2.3 Desafio — AP para palíndromos (L_3)	100% correto, justificativa formal incluída. (0,7)	Correto mas sem justificativa da diferença. (0,45)	Parcialmente correto. (0,2)	Ausente. (0)
Correto para todos os casos de teste de L_3 .				
2.4 Corretude para L_2	100% correto. (0,5)	70–99% correto. (0,3)	Menos de 70%. (0,1)	Não executa. (0)
Todos os casos de teste de $a^n b^n$ produzem resultado esperado.				

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
2.5 Questões reflexivas Lema do Bombeamento, equivalência dos modos de aceitação, papel de Z_0 .	Respostas formais e completas. (0,7)	Corretas mas informais. (0,45)	Superficiais ou incompletas. (0,2)	Ausente. (0)
Subtotal máximo — Parte 2:				3,0

Rubrica — Parte 3: Máquina de Turing (3,0 pontos)

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
3.1 Fidelidade à 7-tupla e estrutura da fita Fita dinâmica, cabeçote, alfabeto com branco.	7-tupla completa, fita implementada corretamente. (0,6)	Pequenas inconsistências de representação. (0,4)	Fita como array fixo ou sem cabeçote explícito. (0,2)	Não imple- menta. (0)
3.2 Corretude para $L_4 = a^n b^n c^n$ Todos os casos de teste produzem resultado esperado.	100% correto, rastreo exibido. (0,6)	70–99% correto. (0,4)	Menos de 70%. (0,2)	Não executa. (0)
3.3 Desafio — MT computadora de $f(n) = n + 1$ Fita de saída contém exatamente $n + 1$ símbolos 1.	Correto para todos os casos de teste. (0,6)	Correto para casos simples, falha em casos maiores. (0,4)	Lógica parcialmente correta. (0,2)	Ausente. (0)
3.4 Exibição passo a passo e limite de passos Cada configuração exibida; limite funcional.	Visualização clara, limite configurável e funcional. (0,5)	Visualização presente mas limite fixo no código. (0,3)	Visualização parcial. (0,1)	Ausente. (0)
3.5 Questões reflexivas Limites do AP, estratégia de marcação, Tese de Church-Turing.	Respostas formais, profundas e corretas. (0,7)	Corretas mas sem formalismo. (0,45)	Superficiais. (0,2)	Ausente. (0)
Subtotal máximo — Parte 3:				3,0

Rubrica — Relatório Técnico e Vídeo de Defesa (1,5 ponto)

Critério	Excelente	Satisfatório	Insuficiente	Não Real.
4.1 Qualidade do relatório Estrutura, linguagem técnica, referências.	Todas as seções presentes, linguagem técnica, ≥ 3 referências corretas. (0,5)	Maioria das seções, linguagem adequada. (0,3)	Seções faltantes ou linguagem inadequada. (0,15)	Ausente. (0)
4.2 Fundamentação teórica no relatório Cálculo manual correto para ao menos 2 cadeias por máquina.	Cálculos formais, corretos e bem apresentados. (0,5)	Cálculos presentes com pequenos erros. (0,3)	Cálculos ausentes ou incorretos. (0,1)	Ausente. (0)
4.3 Vídeo de defesa O aluno demonstra compreensão do próprio código e da teoria; executa os casos de teste ao vivo.	Explica com clareza e precisão, executa todos os testes exigidos, responde às questões reflexivas. (0,5)	Explica a maioria com pequenas lacunas, executa a maior parte dos testes. (0,3)	Dificuldade em explicar decisões; executa apenas parte dos testes. (0,1)	Vídeo ausente ou inacessível. (0)
Subtotal máximo — Parte 4:				1,5

Quadro-Resumo de Pontuação

Componente	Pontuação Máxima	Nota Obtida
Parte 1 — AFD	2,5	
Parte 2 — Autômato de Pilha	3,0	
Parte 3 — Máquina de Turing	3,0	
Parte 4 — Relatório e Vídeo de Defesa	1,5	
TOTAL	10,0	

6. Orientações Finais e Dicas

Como ter sucesso neste trabalho

1. **Comece pela teoria, não pelo código.** Desenhe a 5-tupla (ou 7-tupla) no papel antes de abrir o editor. Se o modelo formal estiver errado, o código também estará.
2. **Teste incrementalmente.** Valide cada caso de teste antes de avançar para o próximo. Uma MT com 10 estados é difícil de depurar se testada apenas ao final.
3. **Use o rastreio a seu favor.** A exibição de configurações instantâneas não é apenas exigência do trabalho — é sua principal ferramenta de depuração.
4. **Documente as transições.** Para cada transição $\delta(q, a) = q'$ (ou equivalente), escreva em comentário o que ela representa semanticamente.
5. **Sobre IA e consultas externas:** ferramentas de IA podem ser usadas para esclarecer dúvidas conceituais, entender erros de compilação ou explorar recursos da linguagem C#. O importante é que o modelo das máquinas abstratas, as decisões de projeto e a escrita do relatório sejam de autoria da equipe. Lembre-se: o vídeo de defesa é o momento em que essa autoria fica evidente.
6. **Commits frequentes no GitHub.** Além de boas práticas de desenvolvimento, o histórico de commits é evidência do processo de construção do trabalho ao longo do tempo.
7. **Grave o vídeo com antecedência.** Reserve tempo para testar a captura de tela, o áudio e a edição básica antes do prazo de entrega.

Referências Sugeridas

1. SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage, 2013.
2. HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.
3. MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. 6. ed. Porto Alegre: Bookman, 2010.
4. LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elements of the Theory of Computation*. 2. ed. Upper Saddle River: Prentice Hall, 1998.
5. RICH, Elaine. *Automata, Computability, and Complexity: Theory and Applications*. Upper Saddle River: Prentice Hall, 2008.